



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Laboratorio No. 2: Clases.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Gustavo O. Zamarron De La Garza

Agosto/2026

Tabla de contenido

I.	Objetivo	3
II.	Descripción de la practica.....	3
	Lista de materiales.....	3
III.	Desarrollo de la práctica	4
	Metodología y Arquitectura	4
	Descripción del problema y elementos clave	4
	Datos de entrada y datos de salida	4
	Diagrama de clases.....	5
	Fragmentos de código y visualización de elementos clave	5
	Clase Rectángulo.....	5
	Clase Círculo.....	6
	Clase Triángulo	7
	Clase Calculadora — menú principal y submenús.....	8
	Reflexión y análisis crítico.....	9
	Dificultades técnicas.....	9
	Fracasos, fallas, bugs y errores	9
	Cómo se resolvió.....	10
	Conclusión	10
IV.	Bibliografía.....	10

I. Objetivo

El alumno diseñará clases para modelar el estado y comportamiento de entidades del mundo real, a través de atributos y métodos, respetando los principios de la orientación a objetos y las capacidades del lenguaje Java. En esta práctica se implementaron clase que representan polígonos y una clase principal llamada calculadora la cual implementa las clases a través de un menú interactivo con el usuario.

II. Descripción de la practica

Haz una clase llamada Rectángulo

Haz un constructor en donde se inicializan todos los atributos

Diseña e implementa sus atributos, setters y getters

Diseña e implementa un método para calcular el área

Diseña e implementa un método para calcular su perímetro

Haz una clase llamada Círculo

Haz un constructor en donde se inicializan todos los atributos

Diseña e implementa sus atributos, setters y getters

Diseña e implementa un método para calcular el área

Diseña e implementa un método para calcular su perímetro

Haz una clase llamada Triángulo

Haz un constructor en donde se inicializan todos los atributos

Diseña e implementa sus atributos, setters y getters

Diseña e implementa un método para calcular el área

Diseña e implementa un método para calcular su perímetro

Haz una clase llamada Calculadora

Esta clase no se debe instanciar (aquí va el main)

haz uso de los constructores para crear los objetos de las figuras.

En la función principal Crea un menú que te permita realizar todas las operaciones de la calculadora (área y perímetro por cada figura)

Pruébalo con un ejemplo por cada operación

Sube tu código, En el apartado de taller sube el reporte

Lista de materiales

- Computadora Personal (PC)
- Ambiente de desarrollo. (Puro bloc de notas papito)
- JDK Java Development KitTexto.

III. Desarrollo de la práctica

Metodología y Arquitectura

Descripción del problema y elementos clave

En este laboratorio, gran parte del análisis del problema ya venía resuelto de antemano, ya que la especificación de la práctica indicaba explícitamente qué clases debían crearse (Rectángulo, Círculo, Triángulo y Calculadora), qué atributos debía tener cada una y qué comportamientos debían implementarse: un constructor, sus setters y getters, y dos métodos adicionales para calcular el área y el perímetro. Esto ahorró el proceso de decidir por cuenta propia qué entidades del mundo real modelar, dejando como tarea principal el diseño de la implementación: cómo declarar correctamente los atributos privados, cómo inicializarlos mediante constructores y cómo estructurar un menú que permitiera usar todas las figuras desde un único punto de entrada.

Los elementos clave del diseño son los siguientes:

- Rectángulo, Círculo y Triángulo se implementaron como clases independientes, cada una con sus atributos numéricos privados de tipo double, un constructor vacío, un constructor con parámetros, sus setters y getters, y los métodos `area()` y `perimetro()`.
- Calculadora es la clase que contiene el método `main` y no está pensada para instanciarse: en vez de eso declara como atributos estáticos un objeto de cada figura (`rectangulo`, `circulo`, `triangulo`) y un `Scanner` compartido, de modo que cualquier método estático de menú pueda leer y modificar esos mismos objetos sin necesidad de recibirlos como parámetro.
- El menú se organizó en un menú principal y un submenú por cada figura, en vez de un único menú plano con todas las operaciones mezcladas.

Datos de entrada y datos de salida

Datos de entrada

- Menú principal: número entero para elegir la figura (1 Rectángulo, 2 Círculo, 3 Triángulo, 4 Salir).
- Submenú Rectángulo: número entero para elegir la operación, y valores double para base y altura.
- Submenú Círculo: número entero para elegir la operación, y un valor double para el radio.
- Submenú Triángulo: número entero para elegir la operación, y valores double para base, altura y, al calcular el perímetro, `ladoA` y `ladoB`.

Datos de salida

- Mensajes de confirmación de captura (por ejemplo, "Captura correcta. Base = ...").
- Resultados numéricos de área y perímetro, siempre formateados a dos decimales mediante `printf("%.2f")`.

- Mensajes de navegación del menú (regresar al menú principal, opción no encontrada, salir del programa).

Diagrama de clases

El siguiente diagrama muestra las tres clases modelo (Rectángulo, Círculo y Triángulo) y la clase Calculadora, que las utiliza mediante una relación de dependencia («uses») mostrada con flecha punteada. Calculadora no aparece conectada por herencia ni composición fuerte porque sus referencias a las figuras son estáticas y se comparten entre los distintos submenús.

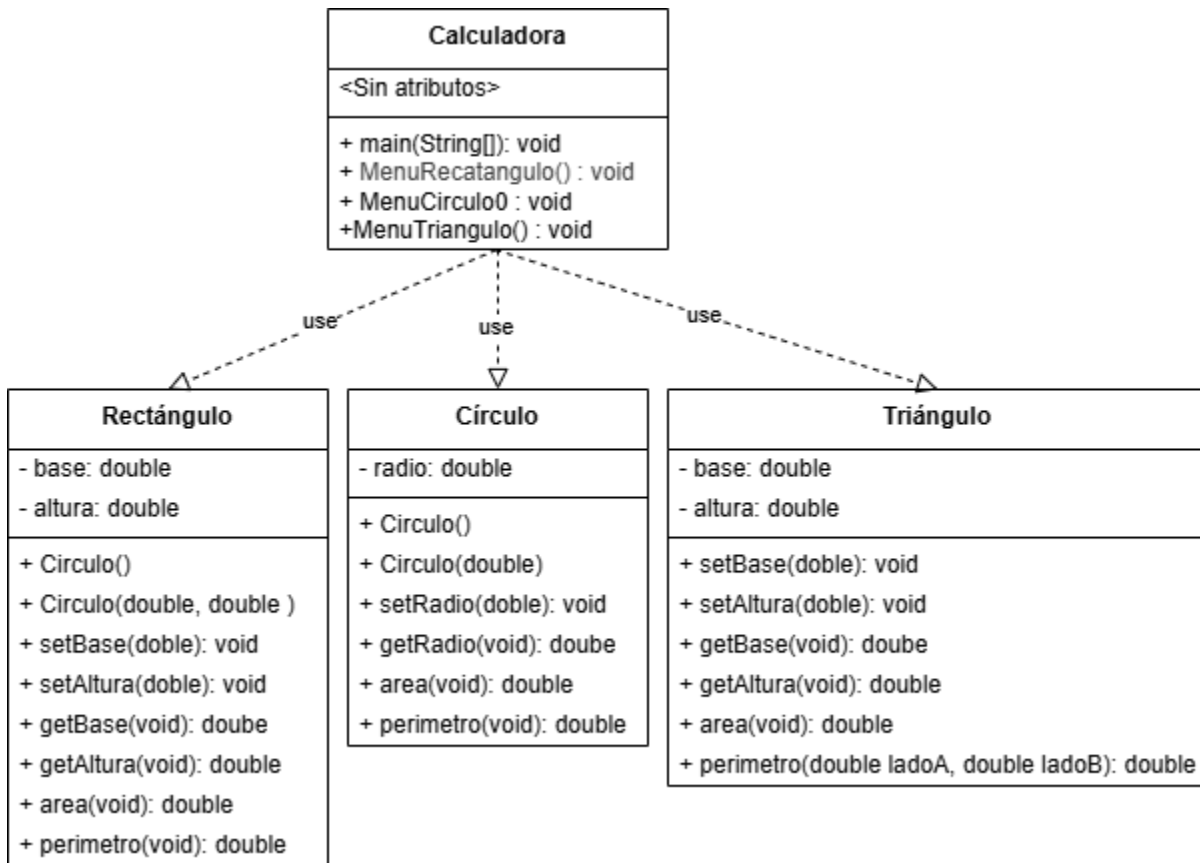


Ilustración 1: Diagrama de clases — Rectángulo, Círculo, Triángulo y Calculadora.

Fragmentos de código y visualización de elementos clave

Clase Rectángulo

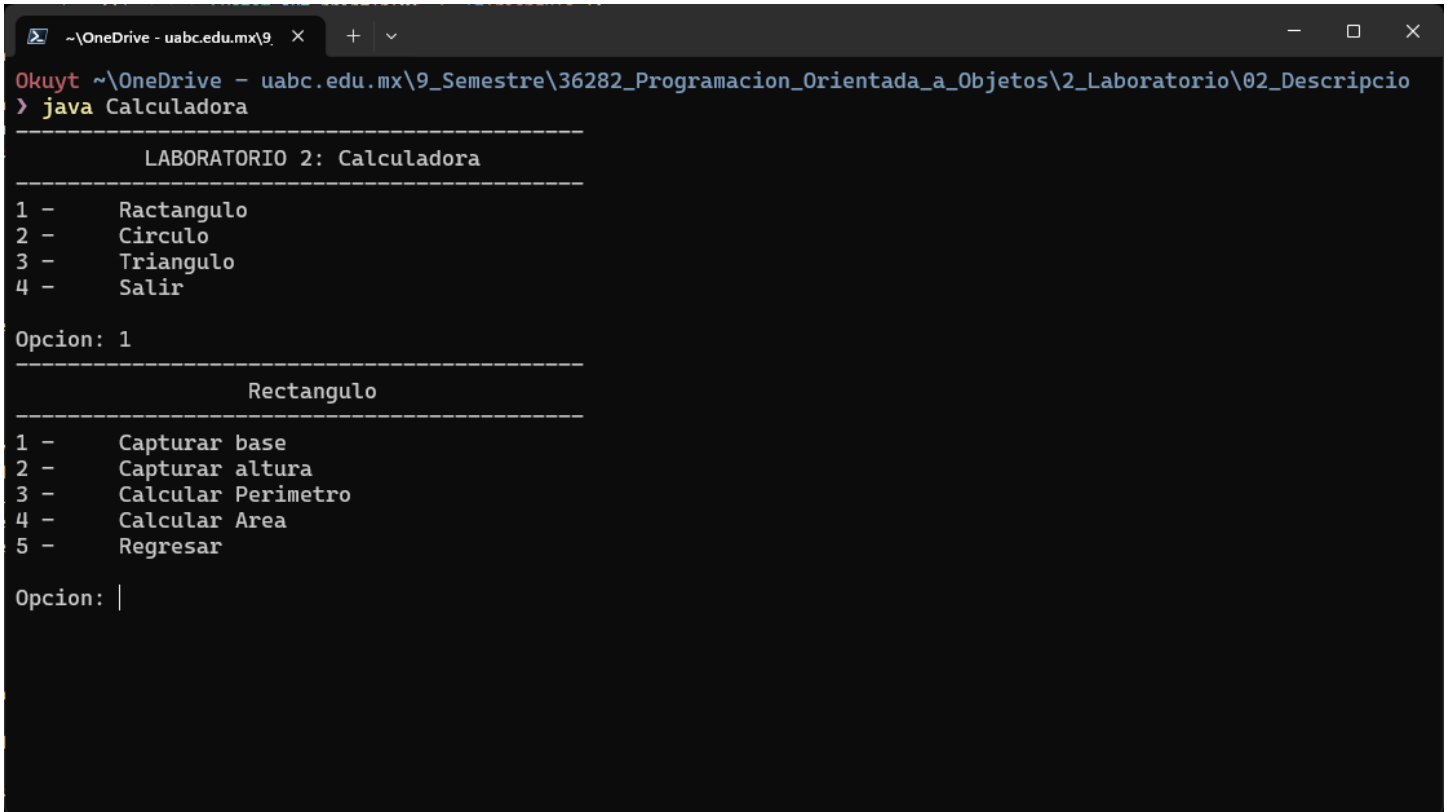
```

public double area(){
    return base * altura;
}

public double perimetro(){ return (base * 2) + (altura * 2); }
  
```

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

Programé area() como una simple multiplicación de base por altura, y perimetro() como la suma de los cuatro lados, agrupando los lados iguales en $2 \cdot \text{base} + 2 \cdot \text{altura}$. Mantuve base y altura como atributos double para poder capturar medidas con decimales, ya que un rectángulo real no siempre tiene medidas enteras.



```
Okuyt ~\OneDrive - uabc.edu.mx\9_ x + v
Okuyt ~\OneDrive - uabc.edu.mx\9_Semestre\36282_Programacion_Orientada_a_Objeto\2_Laboratorio\02_Descripcio
> java Calculadora

-----
LABORATORIO 2: Calculadora
-----
1 - Rectangulo
2 - Circulo
3 - Triangulo
4 - Salir

Opcion: 1
-----
Rectangulo
-----
1 - Capturar base
2 - Capturar altura
3 - Calcular Perimetro
4 - Calcular Area
5 - Regresar

Opcion: |
```

Ilustración 2: Submenú Rectángulo (captura de base/altura y cálculo de área y perímetro)

Clase Círculo

```
public double area(){ return Math.PI * radio; }

public double perimetro(){ return Math.PI * Math.pow(radio, 2); }
```

Para el Círculo usé la constante Math.PI [1] en ambos cálculos a partir del radio, apoyándome en Math.pow() para elevarlo al cuadrado en uno de los dos métodos. El radio se maneja como double por la misma razón que en Rectángulo: permitir medidas con decimales.

```

Okuyt ~\OneDrive - uabc.edu.mx\9 x + v
~\OneDrive - uabc.edu.mx\9_Semestre\36282_Programacion_Orientada_a_Objeto\2_Laboratorio\02_Descripcio
> java Calculadora

-----
LABORATORIO 2: Calculadora
-----
1 - Ractangulo
2 - Circulo
3 - Triangulo
4 - Salir

Opcion: 2
-----
Circulo
-----
1 - Capturar radio
2 - Calcular Perimetro
3 - Calcular Area
4 - Regresar

Opcion: |

```

Ilustración 3: Submenú Círculo (captura de radio y cálculo de área y perímetro)

Clase Triángulo

```

public double area(){ return (base * altura) / 2; }

public double perimetro(double ladoA, double ladoB){
    return ladoA + ladoB + base;
}

```

El área la calculé con la fórmula clásica de base por altura entre dos. Para el perímetro decidí recibir ladoA y ladoB como parámetros del propio método en lugar de agregarlos como atributos de la clase, ya que la especificación sólo pedía base y altura con sus setters y getters; de esta forma el perímetro se calcula sumando esos dos lados capturados en el momento junto con el atributo base ya almacenado en el objeto.

```

> java Calculadora
-----
LABORATORIO 2: Calculadora
-----
1 - Rectangulo
2 - Circulo
3 - Triangulo
4 - Salir

Opcion: 3
-----
Triangulo
-----
1 - Capturar base
2 - Capturar altura
3 - Calcular Perimetro
4 - Calcular Area
5 - Regresar

Opcion: |

```

Ilustración 4: Submenú Triángulo (captura de base/altura, lados A y B, y cálculo de área y perímetro)

Clase Calculadora — menú principal y submenús

```

public static void main (String[] args){
    int opcion;
    do{
        // ...imprime el menú principal...
        opcion = input.nextInt();
        input.nextLine();
        switch(opcion){
            case 1: MenuRectangulo(); break;
            case 2: MenuCirculo();      break;
            case 3: MenuTriangulo();    break;
            case 4: System.out.println("Saliendo..."); break;
            default: System.out.println("No se encontro la opcion");
        }
    } while(opcion != 4);
}

```

Para la Calculadora, específicamente el menú, el diseño fue algo tedioso de decidir: opté por tratarlo como submenús en lugar de un solo menú con todas las operaciones juntas. Así, el menú principal sólo se encarga de dirigir hacia la figura elegida, y cada figura tiene su propio submenú (con su propio ciclo do-while y su propio switch) que agrupa la captura de sus atributos y el cálculo de su área y perímetro. Los objetos rectangulo, circulo y triangulo se declararon como atributos estáticos de la clase para que todos los submenús pudieran compartir la misma instancia sin necesidad de pasarla como

parámetro entre métodos. Además, cada submenú incluye una opción de prueba adicional que no aparece impresa en el texto del menú, pero sí es accesible tecleando su número: crea o ajusta un objeto con valores fijos y muestra sus resultados de una sola vez, lo que me sirvió para verificar rápidamente los cálculos sin tener que capturar todo manualmente cada vez.

```

> java Calculadora
-----
LABORATORIO 2: Calculadora
-----
1 - Rectangulo
2 - Circulo
3 - Triangulo
4 - Salir
Opcion: |

```

Ilustración 5: Menú principal de la Calculadora

Reflexión y análisis crítico

Dificultades técnicas

La dificultad técnica más importante durante este laboratorio fue el cambio de editor de texto: dejé de usar Vim para pasar a un editor de texto nativo del entorno Linux/Ubuntu, principalmente por la facilidad de copiar y pegar que me ofrecía frente a Vim, lo cual agilizó bastante mi flujo de trabajo. Otra dificultad fue el diseño de los menús: tuve que pensar de nuevo la lógica para decidir qué objetos convenía declarar como variables estáticas dentro de la clase Calculadora, de manera que todos los submenús pudieran compartirlos sin duplicar instancias ni perder los valores capturados al cambiar de submenú.

Fracasos, fallas, bugs y errores

La mayoría de los errores que tuve fueron de tipo sintáctico: nombres de clases o de variables mal escritos que el compilador señalaba de inmediato. También tuve un problema al combinar en el mismo Scanner la lectura de un entero (`nextInt()`) con la lectura de valores double, ya que `nextInt()` no consume el salto de línea que queda en el buffer y eso podía interferir con la siguiente captura.

Cómo se resolvió

Los errores de sintaxis los resolví compilando constantemente el proyecto: el terminal indica la línea exacta del error, así que sólo tenía que corregir esos pequeños detalles de escritura uno por uno hasta que el programa compilara sin errores. El problema de mezclar entero y double en el Scanner lo resolví agregando `input.nextLine()` justo después de leer un entero, para limpiar el salto de línea pendiente antes de la siguiente lectura y evitar que el programa se saltara una captura.

Conclusión

Esta práctica me ayudó a ejercitar el pensamiento crítico y reflexivo como programador, ya que me obligó a pensar qué atributos son necesarios y característicos de cada clase, y cómo traducir eso en una solución concreta: en este caso, un menú organizado en submenús que resultara fácil de usar y de leer desde la calculadora, cuidando también la forma en que se presenta la información en la consola.

IV. Bibliografía

- [1] Oracle Corporation. (2024). Java SE Documentation. Recuperado de <https://docs.oracle.com/javase/>
- [2] Deitel, P. J., & Deitel, H. M. (2016). Java: How to Program (10th ed.). Pearson Education.